

Introducción a Python para Ingeniería, Datos y Economía

John Leal

2026-09-22

Tabla de contenidos

Introducción a Python para Ingeniería, Datos y Economía	4
Prólogo	4
Cómo usar este libro	4
1 Introducción a Python y Google Colab	5
1.1 ¿Qué es Python y por qué lo usaremos en Google Colab?	5
1.2 Tu primer programa	5
1.3 Python como calculadora	6
1.4 Los errores son tus amigos	7
1.5 Guardar tu trabajo	7
1.6 Ejercicios	7
1.7 Resumen	8
2 Fundamentos: variables y tipos de datos	9
2.1 ¿Qué es una variable?	9
2.2 Reglas para nombrar variables	9
2.3 Los cuatro tipos de datos básicos	10
2.4 Convertir entre tipos	11
2.5 Pedir datos al usuario: <code>input()</code>	11
2.6 Imprimir con formato: las f-strings	12
2.7 Comentarios: notas para humanos	12
2.8 Ejercicios	13
2.9 Resumen	13
3 Control de flujo: decisiones, repeticiones y funciones	14
3.1 Decisiones: <code>if</code> , <code>elif</code> , <code>else</code>	14
3.1.1 Operadores de comparación y lógicos	15
3.2 Repeticiones: el ciclo <code>for</code>	16
3.3 Repeticiones con condición: el ciclo <code>while</code>	17
3.4 Empaquetar: las funciones	17
3.5 Ejercicios	18
3.6 Resumen	18
4 Estructuras de datos: listas, tuplas y diccionarios	20
4.1 Listas: colecciones ordenadas que puedes modificar	20
4.1.1 Acceder a un elemento: los índices empiezan en cero	20

4.1.2	Modificar, agregar y contar	20
4.1.3	Operaciones útiles sin escribir ciclos	21
4.1.4	Rebanadas (slicing): tomar una parte	21
4.1.5	Recorrer una lista	21
4.2	Tuplas: colecciones que no deben cambiar	22
4.3	Diccionarios: datos con nombre, no con posición	22
4.3.1	Un diccionario puede guardar una ficha completa	23
4.4	¿Cuál estructura usar?	23
4.5	Ejercicios	24
4.6	Resumen	24
5	Capítulo 5: NumPy y gráficas	25
6	Capítulo 6: pandas	26
7	Capítulo 7: Economía y administración	27
8	Capítulo 8: Introducción a big data	28

Introducción a Python para Ingeniería, Datos y Economía

John Leal

Prólogo

Este libro está pensado para estudiantes y profesionales de ingeniería, análisis de datos y economía que desean aprender a programar en Python desde cero. No se asume ninguna experiencia previa en programación: los conceptos se introducen de forma progresiva, con ejemplos cercanos a la ingeniería, la estadística y la economía, para que cada tema tenga un contexto claro y aplicable.

Todo el código está diseñado para ejecutarse en Google Colab, el entorno gratuito de notebooks de Google. No necesitas instalar nada en tu computadora: basta un navegador y una cuenta de Google. Cada capítulo contiene bloques de código listos para copiar y pegar en una celda de Colab, de modo que puedes leer, ejecutar y experimentar en el mismo momento.

Al terminar este libro habrás aprendido los fundamentos del lenguaje Python —variables, tipos de datos, operadores y control de flujo—, las estructuras de datos esenciales, el cálculo numérico con NumPy, la visualización de gráficas, el análisis de datos con pandas, y una introducción a las aplicaciones en economía, administración y big data.

Cómo usar este libro

Cada bloque de código de este libro se copia y se pega tal cual en una celda de Google Colab. Para ello, abre colab.research.google.com, crea un notebook nuevo y copia los bloques de cada capítulo en orden, ejecutándolos con el botón de reproducción (o con **Ctrl + Enter**).

Te recomendamos crear un notebook por capítulo, ejecutar cada bloque paso a paso y modificar los valores de los ejemplos para comprobar qué ocurre: aprender a programar se logra escribiendo y ejecutando código, no solo leyéndolo.

1 Introducción a Python y Google Colab

1.1. ¿Qué es Python y por qué lo usaremos en Google Colab?

Python es un lenguaje de programación de propósito general, creado con una idea central: que el código se lea casi como texto en inglés. Esa claridad lo convirtió en el lenguaje más usado del mundo en ciencia de datos, ingeniería, economía e inteligencia artificial.

¿Por qué Python y no otro lenguaje? Porque con las mismas diez líneas de código que en otros lenguajes apenas configurarían el entorno, en Python ya estás analizando datos reales, graficando una función o calculando el valor presente neto de un proyecto.

En este libro trabajaremos con **Google Colab**: un entorno gratuito de Google que corre en el navegador. No necesitas instalar nada en tu computador; solo una cuenta de Google. Cada bloque de código de este libro está diseñado para que lo copies, lo pegues en una celda de Colab y lo ejecutes.

1.2. Tu primer programa

1. Abre colab.research.google.com en tu navegador.
2. Haz clic en **Nuevo notebook**.
3. En la primera celda, escribe (o copia) lo siguiente:

```
print("Hola, mundo")
```

4. Ejecuta la celda con el botón  o con las teclas **Shift + Enter**.

En pantalla aparecerá:

```
Hola, mundo
```

¿Qué acaba de pasar? `print()` es una **función**: una instrucción que le dice a Python “muestra esto en pantalla”. El texto entre comillas se llama **cadena de caracteres** (*string*). Acabas de escribir y ejecutar tu primer programa.

Nota

Un notebook de Colab tiene dos tipos de celdas: **código** (se ejecuta) y **texto** (se lee, como este párrafo). Agrégalas con los botones + Código y + Texto.

1.3. Python como calculadora

Antes de programar “en serio”, usa Python como una calculadora. Copia y ejecuta cada línea:

```
2 + 3
```

```
10 / 4
```

```
2 ** 10
```

```
17 % 5
```

Los operadores básicos son:

Tabla 1.1: Operadores aritméticos básicos de Python

Operador	Significado	Ejemplo	Resultado
+	Suma	2 + 3	5
-	Resta	7 - 4	3
*	Multiplicación	3 * 5	15
/	División	10 / 4	2.5
**	Potencia	2 ** 10	1024
//	División entera	17 // 5	3
%	Residuo (módulo)	17 % 5	2

Un ejemplo con sentido económico: si inviertes \$1.000.000 a una tasa del 12% anual durante 5 años, el valor futuro es:

```
1000000 * (1 + 0.12) ** 5
```

Observa que Python respeta el orden de las operaciones (primero paréntesis, luego potencias, luego multiplicación y división, al final suma y resta), igual que en álgebra.

1.4. Los errores son tus amigos

Escribe esto en una celda y ejecútalo:

```
print("Hola"
```

Verás algo como:

```
SyntaxError: incomplete input
```

No te alarmes: un error no rompe nada. Python te está diciendo **qué** falló (**SyntaxError**: error de sintaxis) y en este caso es evidente: falta cerrar el paréntesis. Corrige y ejecuta de nuevo.

Tip

Regla de oro para todo el libro: **lee los errores de abajo hacia arriba**. La última línea del mensaje casi siempre dice exactamente qué salió mal.

1.5. Guardar tu trabajo

Colab guarda tus notebooks automáticamente en tu Google Drive (carpeta Colab Notebooks). Buenas prácticas desde el primer día:

- Renombra el notebook: clic en el título de arriba a la izquierda. Usa nombres sin espacios ni tildes: `capitulo01_introduccion.ipynb`.
- Para descargarlo: Archivo → Descargar → Descargar `.ipynb`.
- Crea un notebook por capítulo de este libro; te servirán como tu propio cuaderno de apuntes ejecutable.

1.6. Ejercicios

1. Ejecuta `print("Tu nombre")` con tu nombre real.
2. Calcula cuántos segundos tiene un año (365 días) usando solo operadores.
3. Calcula el valor futuro de \$5.000.000 al 8% anual durante 10 años.
4. Provoca a propósito un error quitando una comilla en un `print`, lee el mensaje y corrígelo.
5. **Mini-reto:** una hectárea produce 4,2 toneladas de café. ¿Cuántas toneladas producen 37 hectáreas? ¿Y si el precio es \$9.500.000 por tonelada, cuánto vale la cosecha total? Resuélvelo en una sola celda con dos operaciones.

1.7. Resumen

- Python es un lenguaje claro y es el estándar en datos e ingeniería.
- Colab te permite programar sin instalar nada, desde el navegador.
- `print()` muestra resultados en pantalla.
- Python funciona como calculadora con los operadores `+` `-` `*` `/` `**` `//` `%`.
- Los errores son mensajes de ayuda: léelos de abajo hacia arriba.

2 Fundamentos: variables y tipos de datos

2.1. ¿Qué es una variable?

Imagina una caja con una etiqueta. Dentro de la caja guardas algo (un número, un texto), y la etiqueta te permite encontrarla después. Eso es una **variable**: un nombre que guarda un valor.

En Python, guardar un valor en una variable se llama **asignar**, y se hace con el signo =:

```
precio = 9500
```

Lee esa línea así: “la variable `precio` guarda el valor 9500”. Ojo: el = de Python **no es** el igual de las matemáticas; no afirma que dos cosas sean iguales, sino que **guarda** el valor de la derecha en el nombre de la izquierda.

A partir de ahí, puedes usar la variable en cualquier cálculo:

```
precio = 9500
cantidad = 37
total = precio * cantidad
print(total)
```

La gran ventaja: si el precio cambia, solo modificas una línea y todo lo demás se recalcula solo. Cambia `precio` a 10200 y vuelve a ejecutar.

2.2. Reglas para nombrar variables

Python es estricto con los nombres. Estos son válidos:

```
precio_tonelada = 9500
temperatura2 = 25.4
nombre_finca = "La Esperanza"
```

Estos **no** son válidos (ejecútalos y lee el error):

```
2temperatura = 25      # no puede empezar con número
precio-tonelada = 9500 # no puede tener guiones
precio tonelada = 9500 # no puede tener espacios
```

i Nota

Convención de este libro: nombres en minúsculas, sin tildes, con palabras separadas por guion bajo: `valor_futuro`, `tasa_anual`, `numero_hectareas`. No es obligatorio, pero es el estilo que usa todo el mundo de Python.

2.3. Los cuatro tipos de datos básicos

Todo valor en Python tiene un **tipo**. Para empezar solo necesitas conocer cuatro:

Tipo	Nombre	Qué guarda	Ejemplo
<code>int</code>	Entero	Números sin decimales	37, -5, 1000000
<code>float</code>	Decimal	Números con decimales	4.2, 0.12, -3.5
<code>str</code>	Texto	Cadenas de caracteres	"café", "La Esperanza"
<code>bool</code>	Booleano	Verdadero o falso	True, False

Python detecta el tipo automáticamente según lo que guardes. Puedes preguntárselo con la función `type()`:

```
precio = 9500
print(type(precio))
```

```
rendimiento = 4.2
print(type(rendimiento))
```

```
nombre = "Ana"
print(type(nombre))
```

```
es_rentable = True
print(type(es_rentable))
```

Advertencia

Cuidado con el punto y la coma: en Python los decimales se escriben con **punto**, no con coma. Escribe 4.2, no 4,2. Si escribes 4,2 Python no dará un error obvio: pensará que son dos números separados. Es uno de los errores más comunes de quienes empiezan en países hispanos.

2.4. Convertir entre tipos

A veces necesitas transformar un tipo en otro. Se hace con funciones que tienen el nombre del tipo: `int()`, `float()`, `str()`.

El caso clásico: los números que vienen como texto no se pueden sumar:

```
a = "2"
b = "3"
print(a + b)
```

¿Sorprendido? El resultado es "23", no 5. Como `a` y `b` son **texto**, el `+` los une en lugar de sumarlos (pegar textos se llama **concatenar**). La solución es convertirlos primero:

```
a = "2"
b = "3"
print(int(a) + int(b))
```

2.5. Pedir datos al usuario: `input()`

Hasta ahora los valores estaban escritos en el código. Pero normalmente querrás preguntarle a la persona que usa tu programa. Eso se hace con `input()`:

```
nombre = input("¿Cómo te llamas? ")
print("Hola,", nombre)
```

Al ejecutar, el programa se detiene, muestra la pregunta y espera a que escribas algo y presiones **Enter**.

! Importante

Lo que `input()` devuelve siempre es texto (`str`), aunque la persona escriba un número. Si vas a hacer cuentas con esa respuesta, conviértela:

```
edad = input("¿Cuántos años tienes? ")
edad = int(edad)
print("El próximo año tendrás", edad + 1)
```

O en una sola línea, que es como lo verás en la práctica:

```
edad = int(input("¿Cuántos años tienes? "))
```

2.6. Imprimir con formato: las f-strings

Ya sabes que `print()` muestra cosas. Cuando quieras mezclar texto con variables de forma elegante, usa una **f-string**: una cadena con una `f` antes de las comillas, donde las variables van entre llaves `{}`:

```
precio = 9500
cantidad = 37
total = precio * cantidad
print(f"El total de la cosecha es ${total}")
```

Todo lo que está dentro de las llaves se evalúa. Puedes incluso hacer cuentas ahí dentro:

```
tasa = 0.12
capital = 1000000
print(f"En 5 años tendrás ${capital * (1 + tasa) ** 5}")
```

2.7. Comentarios: notas para humanos

Todo lo que escribas después de un `#` en una línea, Python lo ignora por completo. Sirve para dejar notas que expliquen tu código:

```
# Cálculo del valor futuro a 5 años
tasa = 0.12          # tasa anual
capital = 1000000    # inversión inicial en pesos
```

```
vf = capital * (1 + tasa) ** 5
print(f"Valor futuro: ${vf}")
```

Comenta tu código pensando en la persona que lo leerá en seis meses. Esa persona, casi siempre, serás tú.

2.8. Ejercicios

1. Crea tres variables con tu nombre, tu edad y tu estatura (en metros, con decimal). Imprímelas con una f-string en una sola frase.
2. Pregunta al usuario cuántas hectáreas tiene su finca y cuántas toneladas produce por hectárea. Calcula e imprime la producción total.
3. Pregunta al usuario su salario mensual y calcula cuánto gana al año. Imprime el resultado con una f-string.
4. Ejecuta esto, lee el error y corrígelo:

```
precio = input("Precio: ")
print(precio * 2)
```

(Pista: multiplica texto por dos y verás algo raro, pero no es lo que queremos).

5. **Mini-reto (economía):** pide al usuario un capital inicial, una tasa de interés anual (en porcentaje, ej. 12) y un número de años. Calcula el valor futuro con interés compuesto e imprímelo con formato: **Con \$X al Y% durante Z años, tendrás \$W.** Recuerda convertir la tasa de porcentaje a decimal dividiendo entre 100.

2.9. Resumen

- Una **variable** es un nombre que guarda un valor; se asigna con `=`.
- Los cuatro tipos básicos: `int` (enteros), `float` (decimales), `str` (texto), `bool` (verdadero/falso).
- Los decimales se escriben con **punto**: `4.2`.
- `input()` siempre devuelve texto; usa `int()` o `float()` para hacer cuentas.
- Las **f-strings** (`f"...{variable}..."`) mezclan texto y valores fácilmente.
- Los **comentarios** con `#` documentan tu código.

3 Control de flujo: decisiones, repeticiones y funciones

Hasta ahora tus programas ejecutan las líneas una tras otra, de arriba hacia abajo, siempre igual. Los programas útiles hacen tres cosas más: **deciden** (si pasa esto, haz aquello), **repiten** (haz esto 1000 veces) y **empaquetan** (guarda este procedimiento para usarlo cuando quiera). Eso es el control de flujo.

3.1. Decisiones: if, elif, else

Un sistema de riego automático tiene una lógica muy simple: *si la humedad del suelo es baja, activar la bomba*. En Python eso se escribe casi igual que en español:

```
humedad = 25 # porcentaje medido por el sensor

if humedad < 30:
    print("Humedad baja: activar riego")
```

Ejecútalo. Luego cambia `humedad` a 45 y ejecútalo de nuevo: no aparece nada, porque la condición ya no se cumple.

Para cubrir más casos se usan `elif` (“si no, entonces si...”) y `else` (“en cualquier otro caso”):

```
humedad = 25

if humedad < 30:
    print("Humedad baja: activar riego")
elif humedad < 60:
    print("Humedad adecuada: no hacer nada")
else:
    print("Humedad excesiva: revisar drenaje")
```

⚠ Advertencia

La indentación manda. Fíjate que las líneas después de `if` van desplazadas 4 espacios a la derecha. Ese sangrado le dice a Python qué pertenece a cada bloque. Si lo quitas, da error. Es la regla más famosa de Python: la sangría no es decoración, es sintaxis.

3.1.1. Operadores de comparación y lógicos

Las condiciones se construyen con estos operadores:

Tabla 3.1: Operadores de comparación y lógicos

Tipo	Operador	Significado	Ejemplo	Resultado
Comparación	<code>==</code>	Igual a	<code>5 == 5</code>	<code>True</code>
Comparación	<code>!=</code>	Diferente de	<code>5 != 3</code>	<code>True</code>
Comparación	<code><</code>	Menor que	<code>25 < 30</code>	<code>True</code>
Comparación	<code>></code>	Mayor que	<code>7 > 4</code>	<code>True</code>
Comparación	<code><=</code>	Menor o igual	<code>3 <= 2</code>	<code>False</code>
Comparación	<code>>=</code>	Mayor o igual	<code>30 >= 30</code>	<code>True</code>
Lógico	<code>and</code>	Y (ambas verdaderas)	<code>True and False</code>	<code>False</code>
Lógico	<code>or</code>	O (al menos una)	<code>True or False</code>	<code>True</code>
Lógico	<code>not</code>	Negación	<code>not True</code>	<code>False</code>

! Importante

Doble igual para comparar. Es el error número 1 de los principiantes: `=` asigna, `==` compara. `if humedad = 30:` es error; `if humedad == 30:` es correcto.

Con `and` y `or` puedes combinar condiciones, como una alarma de invernadero que se activa por temperatura **o** por humedad:

```
temperatura = 38
humedad = 45

if temperatura > 35 or humedad > 80:
    print("ALARMA: condiciones fuera de rango")
else:
    print("Condiciones normales")
```

3.2. Repeticiones: el ciclo for

Un sensor toma una lectura cada hora. Un administrador calcula la nómina de 50 empleados. Un ingeniero evalúa 12 parcelas. En todos los casos: **la misma operación, muchas veces**. Para eso está el ciclo `for`.

La forma más simple usa `range()`, que genera una secuencia de números:

```
for hora in range(24):
    print(f"Tomando lectura de la hora {hora}")
```

Esto ejecuta el `print` 24 veces, con `hora` tomando los valores 0, 1, 2, ..., 23.

Ejemplo agro: calcular la producción total recorriendo las parcelas de una finca:

```
producciones = [4.2, 3.8, 5.1, 4.6, 3.9] # toneladas por parcela
total = 0

for p in producciones:
    total = total + p

print(f"Producción total: {total} toneladas")
```

El patrón `total = total + p` se llama **acumulador**: una variable que empieza en cero y va sumando. Lo usarás constantemente.

Ejemplo de administración: proyectar las ventas de los próximos 5 años con crecimiento del 8% anual:

```
ventas = 50000000 # ventas del año actual

for año in range(1, 6):
    ventas = ventas * 1.08
    print(f"Año {año}: ${ventas:,.0f}")
```

Fíjate en el truco `{ventas:,.0f}` dentro de la f-string: formatea el número con separador de miles y sin decimales.

3.3. Repeticiones con condición: el ciclo while

El `for` repite un número conocido de veces. El `while` repite **mientras una condición sea verdadera** — cuando no sabes cuántas vueltas harán falta:

```
tanque = 0      # litros
capacidad = 1000

while tanque < capacidad:
    tanque = tanque + 150    # la bomba llena 150 litros por ciclo
    print(f"Nivel actual: {tanque} litros")

print("Tanque lleno")
```

Advertencia

Si la condición de un `while` jamás se hace falsa, el ciclo corre para siempre (ciclo infinito). En Colab lo detienes con el botón . Asegúrate siempre de que algo dentro del ciclo acerque la condición a su fin.

3.4. Empaquetar: las funciones

Ya usaste funciones que vienen con Python: `print()`, `input()`, `int()`. Ahora crearás las tuyas. Una función es un procedimiento con nombre: le das entradas (**parámetros**), hace su trabajo y te devuelve un resultado (**retorno**).

```
def valor_futuro(capital, tasa, años):
    vf = capital * (1 + tasa) ** años
    return vf
```

- `def` le dice a Python “voy a definir una función”.
- Entre paréntesis van los parámetros: lo que la función necesita recibir.
- `return` entrega el resultado y termina la función.

Se usa así, todas las veces que quieras:

```
inversion1 = valor_futuro(1000000, 0.12, 5)
inversion2 = valor_futuro(5000000, 0.08, 10)

print(f"Proyecto A: ${inversion1:,.0f}")
print(f"Proyecto B: ${inversion2:,.0f}")
```

La ventaja enorme: la fórmula se escribe **una sola vez**. Si mañana hay que corregirla, se corrige en un solo lugar.

Otro ejemplo, ahora de mecatrónica: una función que clasifica la lectura de un sensor de temperatura:

```
def clasificar_temperatura(t):  
    if t < 15:  
        return "frío"  
    elif t <= 30:  
        return "óptimo"  
    else:  
        return "caliente"  
  
print(clasificar_temperatura(22))  
print(clasificar_temperatura(38))
```

3.5. Ejercicios

1. Escribe un `if` que imprima “Crédito aprobado” si el puntaje crediticio es mayor o igual a 700, y “Crédito rechazado” en caso contrario. Prueba con 650 y con 720.
2. Un motor debe apagarse si la temperatura supera 90 °C y el voltaje es inestable (variable booleana `voltaje_inestable`). Escribe la condición con `and` y pruébala con distintos valores.
3. Con un `for` y un acumulador, suma los costos de una lista de insumos: [120000, 85000, 210000, 45000]. Imprime el total.
4. Escribe una función `area_parcela(largo, ancho)` que devuelva el área en metros cuadrados, y úsala para tres parcelas distintas.
5. Escribe una función `es_rentable(ingresos, costos)` que devuelva `True` si los ingresos superan a los costos y `False` si no.
6. **Mini-reto (agro + economía):** tienes las producciones de 6 parcelas: [4.2, 2.8, 5.1, 3.3, 4.6, 2.1] toneladas. Con un `for` y un contador, calcula cuántas parcelas produjeron menos de 3 toneladas (bajo rendimiento) e imprime el porcentaje que representan del total. Luego empaqueta tu solución en una función `porcentaje_bajo_rendimiento(producciones, umbral)`.

3.6. Resumen

- `if` / `elif` / `else` ejecutan bloques según condiciones; la **indentación define los bloques**.

- Comparar con `==`, asignar con `=`. Combinar condiciones con `and`, `or`, `not`.
- `for` repite un número conocido de veces; `while` repite mientras la condición sea verdadera.
- El **acumulador** (`total = total + x`) es el patrón básico para sumar en ciclos.
- Las **funciones** (`def ... return`) empaquetan procedimientos para reutilizarlos sin duplicar código.

4 Estructuras de datos: listas, tuplas y diccionarios

Una variable guarda **un** valor. Pero en la práctica manejas colecciones: las lecturas de un sensor durante el día, los precios de 20 productos, las coordenadas de las parcelas. Para eso existen las estructuras de datos. En este capítulo verás las tres esenciales: **listas**, **tuplas** y **diccionarios**.

4.1. Listas: colecciones ordenadas que puedes modificar

Una lista es una secuencia de valores entre corchetes [], separados por comas:

```
lecturas = [22.5, 23.1, 24.0, 23.8, 25.2] # temperaturas de un sensor
```

4.1.1. Acceder a un elemento: los índices empiezan en cero

Cada elemento tiene una posición llamada **índice**. La regla que confunde a todo principiante: el **primer elemento es el índice 0**, no el 1:

```
lecturas = [22.5, 23.1, 24.0, 23.8, 25.2]

print(lecturas[0]) # primera lectura
print(lecturas[2]) # tercera lectura
print(lecturas[-1]) # última lectura (el -1 cuenta desde el final)
```

4.1.2. Modificar, agregar y contar

Las listas son **mutables**: puedes cambiarlas después de crearlas:

```
precios = [9500, 8200, 10500]

precios[0] = 9800      # modificar: el primer precio cambió
precios.append(7600)   # agregar al final
print(precios)
print(len(precios))    # cuántos elementos tiene
```

4.1.3. Operaciones útiles sin escribir ciclos

Python trae funciones que resumen una lista en una línea:

```
producciones = [4.2, 3.8, 5.1, 4.6, 3.9]

print(sum(producciones))    # total
print(max(producciones))    # la mayor
print(min(producciones))    # la menor
print(len(producciones))    # cuántas parcelas
print(sum(producciones) / len(producciones)) # promedio
```

4.1.4. Rebanadas (slicing): tomar una parte

Con [inicio:fin] tomas un pedazo de la lista (incluye el inicio, **no** incluye el fin):

```
lecturas = [22.5, 23.1, 24.0, 23.8, 25.2]

print(lecturas[1:3])    # del índice 1 al 2
print(lecturas[:2])     # las dos primeras
print(lecturas[3:])     # del índice 3 al final
```

4.1.5. Recorrer una lista

Ya lo hiciste en el capítulo anterior: el `for` nació para esto:

```
ventas = [12.5, 15.2, 9.8, 18.1] # millones por trimestre

for v in ventas:
    if v < 10:
        print(f"Trimestre bajo: {v}")
```

4.2. Tuplas: colecciones que no deben cambiar

Una tupla se escribe con paréntesis () y es **immutable**: una vez creada, no se puede modificar. Sirve para datos que conceptualmente van juntos y no deben alterarse, como las coordenadas de una parcela:

```
ubicacion = (3.42, -76.53) # latitud, longitud  
  
print(ubicacion[0]) # latitud
```

Si intentas modificarla, Python te protege con un error:

```
ubicacion[0] = 4.0 # TypeError: las tuplas no se modifican
```

Regla práctica: si los datos representan algo fijo (coordenadas, una fecha, un registro que no debe tocarse), usa tupla. Si van a cambiar (lecturas, precios, inventarios), usa lista.

4.3. Diccionarios: datos con nombre, no con posición

En una lista accedes por posición (precios[0]). Pero ¿y si quieres acceder **por nombre**? Para eso están los diccionarios: pares de **clave** y **valor** entre llaves {}:

```
precios = {  
    "café": 9500,  
    "cacao": 8200,  
    "banano": 7600  
}  
  
print(precios["café"]) # acceder por clave
```

Agregar y modificar es asignar a una clave:

```
precios["banano"] = 7800 # modificar  
precios["panela"] = 5200 # agregar un producto nuevo  
print(precios)
```

Recorrer un diccionario con `.items()` te da clave y valor a la vez:

```

inventario = {"motobombas": 3, "sensores_humedad": 12, "valvulas": 25}

for equipo, cantidad in inventario.items():
    if cantidad < 5:
        print(f"Reponer {equipo}: quedan {cantidad}")

```

4.3.1. Un diccionario puede guardar una ficha completa

Los diccionarios brillan cuando un valor es a su vez otra estructura:

```

empleado = {
    "nombre": "María Pérez",
    "cargo": "Analista",
    "salario": 2800000,
    "areas": ["compras", "logística"]
}

print(empleado["nombre"])
print(empleado["salario"] * 12)    # salario anual

```

Este patrón — una ficha con campos nombrados — es la base de casi todo el manejo de datos que verás en el capítulo de pandas.

4.4. ¿Cuál estructura usar?

Estructura	Sintaxis	¿Se modifica?	Acceso por	Úsala para
Lista	[1, 2, 3]	Sí	Posición	Series de mediciones, colecciones que crecen
Tupla	(1, 2)	No	Posición	Datos fijos que van juntos (coordenadas)
Diccionario	{"a": 1}	Sí	Nombre (clave)	Fichas, inventarios, catálogos, configuración

4.5. Ejercicios

1. Crea una lista con las temperaturas de 7 días. Imprime la máxima, la mínima y el promedio (con `sum` y `len`).
2. De la lista `costos = [45000, 120000, 38000, 90000, 67000]`, imprime solo los costos mayores a 50000 usando un `for` y un `if`.
3. Crea una tupla con la coordenada de tu ciudad y otra con la de una finca. Imprime la distancia en latitud (diferencia simple del primer elemento).
4. Crea un diccionario con 4 productos agrícolas y sus precios. Sube todos los precios un 10% recorriéndolo con `.items()` y reasignando cada valor.
5. Crea un diccionario que represente un sensor con las claves `id`, `tipo`, `parcela` y `lecturas` (una lista de 5 valores). Imprime el promedio de sus lecturas accediendo a través del diccionario.
6. **Mini-reto (administración):** tienes esta lista de ventas mensuales (millones): `[42, 38, 55, 61, 47, 39, 52, 58, 63, 49, 44, 57]`. Sin usar funciones hechas, solo un `for` y acumuladores: calcula el promedio anual, cuenta cuántos meses superaron el promedio e imprime ambos resultados. Luego, guarda en un diccionario `resumen` las claves `"promedio"` y `"meses_sobre_promedio"`.

4.6. Resumen

- **Lista** `[]`: ordenada y modificable; el caballo de batalla. Índices desde `0`.
- `append()` agrega, `len()` cuenta, `sum()/max()/min()` resumen, `[i:j]` rebana.
- **Tupla** `()`: igual a la lista pero inmutable; para datos fijos.
- **Diccionario** `{}`: pares clave-valor; acceso por nombre con `d["clave"]`, recorrido con `.items()`.
- Elegir bien la estructura es la mitad del programa.

5 Capítulo 5: NumPy y gráficas

En construcción.

6 Capítulo 6: pandas

En construcción.

7 Capítulo 7: Economía y administración

En construcción.

8 Capítulo 8: Introducción a big data

En construcción.